CMSC477: Robotics Perception and Planning
Project 2: Swap it!

Dr. Troi Williams, Anukriti Singh, Khuzema Habib
March 13th, 2026
Due: April 10th, 2026

We think this project is challenging, we encourage you to try to finish before the deadline.

Introduction:

In this project, you will program a robot to autonomously swap the positions of two stacks of LEGO's that are close to each other and on top of color targets. You will need to use a state machine to switch between grasp, move, and place modes as needed to accomplish the task.

An animated description is given below:

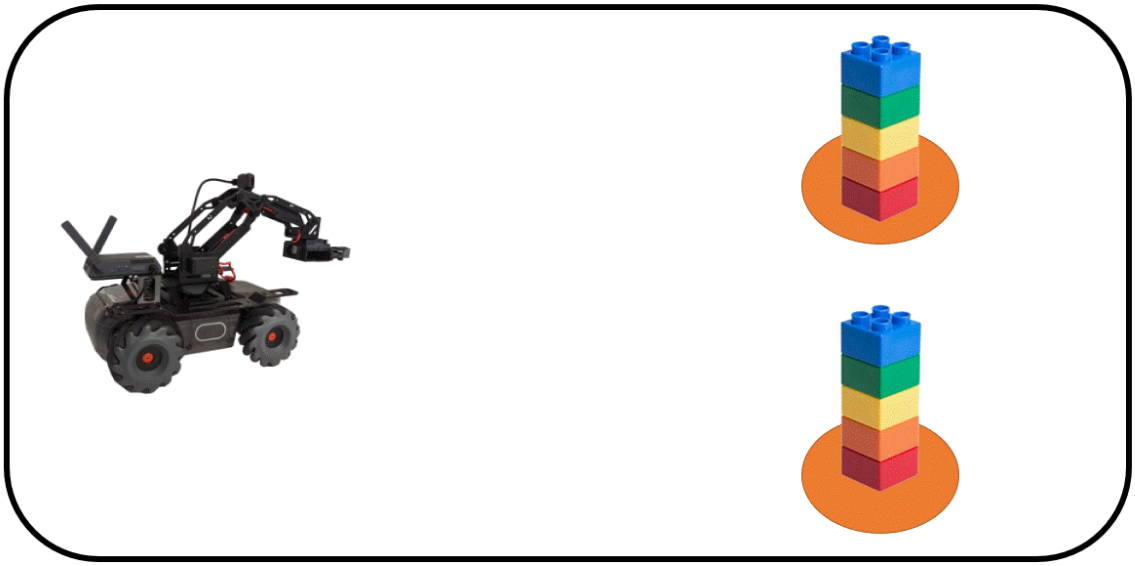

Figure 1: Animation of the swapping goal for Project 2.

Requirements:

- The control sequence cannot be predetermined
    - To show this, the submitted video should show a human moving the targets and the 1st object (which is placed somewhere in free space) while the robot is executing the sequence.
- Do not use AprilTags for the final demonstration
    - You can use AprilTags before the final demo to help "bootstrap" your code

Suggestions:

You are free to accomplish this project in the manner of your choosing. However, we make the following suggestions:

- Detect a bounding box around the target object
    - A fragile but easy-to-implement option is color based segmentation
        - You can implement some basic position filtering to throw out incorrectly identified regions of the image
    - Another option is to use a machine learning algorithm like YOLO
        - See the appendix of this document for an example script that will run a pre-trained model
        - You will need to fine-tune (train) YOLO to detect the LEGO targets
        - Fine-tuning will require you to collect a labeled dataset (50-100 images taken by the RoboMaster camera should be enough)
        - We encourage groups to pool their training sets to improve the overall performance of the class
        - A tutorial slide deck for training YOLO be posted during spring break (March 15-22nd)
- Use the size of the box to determine the target objects' approximate 3D position
    - The most reliable way to do this is likely by using the height of the bounding box in pixels and then estimating the distance with the equation:
      Z = object height / (pixels / camera focal length)
    - You may want to use some averaging or low pass filtering to limit noise
- Check out the robot arm control tutorial which will be posted over spring break
    - The tutorial will show how to work around some quirks in the arm's control and feedback commands
- Adapt the control scheme from Project 1 to control the XY position and heading
- You will need to devise a simple search procedure that finds the LEGOs and reidentifies the targets

Grading:

- Successfully pick up the objects: 15 points
- Successfully recover from a human moving the placement targets: 30 points
- Successfully recover from a human moving the 1st object: 15 points
    - (that is after the robot has left the 1st object to go get the 2nd object)
- Successfully place the objects: 20 points
- Report: 20 points
- Bonus: Consider N different objects and sort them into an order: 10 points

Submission Guidelines:

Please submit a video and PDF report. The video should show your team's demonstration and any relevant animated visualizations. The report should have the following sections:

- Introduction
- Block Diagram
- Link to and description of demonstration video (Google Drive or YouTube)
- Methodology
- Results
- Conclusion
- Appendix A: Code Listing

Collaboration Policy:

You can discuss the assignment with any number of people. But the report and code you turn in MUST be original to your team. Plagiarism is strictly prohibited. A plagiarism checker will be used to check your submission. Please make sure to cite any references from papers, websites, or any other student's work you might have referred to.

Appendix:

To get the machine learning algorithm running using the image from the robot:

- Install the Ultralytics YOLO algorithm package: **"pip install ultralytics"**
    - If you have a laptop with an NVIDIA GPU you can run the machine learning algorithm much faster by installing CUDA enabled PyTorch as instructed by this link: https://pytorch.org/get-started/locally/
- Run the following code:

```
from ultralytics import YOLO
import cv2
import time
from robomaster import robot
from robomaster import camera

print('model')
model = YOLO("yolo11n.pt")

# Use vid instead of ep_camera to use your laptop's webcam
# vid = cv2.VideoCapture(0)

ep_robot = robot.Robot()
ep_robot.initialize(conn_type="ap")
ep_camera = ep_robot.camera
ep_camera.start_video_stream(display=False, resolution=camera.STREAM_360P)
```

```
while True:
    # ret, frame = vid.read()
    frame = ep_camera.read_cv2_image(strategy="newest", timeout=0.5)
    if frame is not None:
        start = time.time()
        if model.predictor:
            model.predictor.args.verbose = False
        result = model.predict(source=frame, show=False)[0]

        # DIY visualization is much faster than show=True for some reason
        boxes = result.boxes
        for box in boxes:
            xyxy = box.xyxy.cpu().numpy().flatten()
            cv2.rectangle(frame,
                          (int(xyxy[0]), int(xyxy[1])),
                          (int(xyxy[2]), int(xyxy[3])),
                           color=(0, 0, 255), thickness=2)

        cv2.imshow('frame', frame)
        key = cv2.waitKey(1)
        if key == ord('q'):
            break

        # print(results)

        end = time.time()
        print(1.0 / (end-start))
```